Formalizing the Christofides–Serdyukov 3/2-Approximation Algorithm and the Held–Karp Petersen Integrality Gap in Lean 4

Ian McNelly
https://github.com/imcn23/tsp

November 2026

Abstract

We describe a Lean 4 formalization of two classical results on the Traveling Salesperson Problem (TSP) over finite undirected graphs: the Christofides–Serdyukov 3/2-approximation guarantee for metric TSP and the polyhedral integrality gap of the Held–Karp subtour elimination relaxation on the 10-vertex Petersen graph P_{10}. In the approximation development (`Christofides.lean`), we represent Hamiltonian tours as duplicate-free cyclic vertex lists over `Fin n` and model undirected trees, matchings, and Eulerian multigraphs using `Finset (Sym2 (Fin n))` and `Multiset (Sym2 (Fin n))`. This representation reduces metric walk shortcutting to structural induction on sublists and supports constructive proofs of the spanning-tree lower bound $\text{cost}(T) \leq \text{OPT}$, the handshaking parity of odd-degree vertices, the alternating-cycle matching bound $2 \cdot \text{cost}(M) \leq \text{OPT}$, and Hierholzer cycle splicing for Eulerian circuits. In the complementary polyhedral development (`TSPComplexity.lean`), we formalize the Dantzig–Fulkerson–Johnson (DFJ) subtour cut inequality via a $\mathbb{Z}_2$ telescoping sum over cyclic shift bijections, embed integral tours into the Held–Karp relaxation, and use kernel computation (`decide`) together with Mathlib's `convexHull_min` to prove that the uniform $x_e = 2/3$ fractional assignment on P_{10} satisfies all 1,022 proper DFJ cut constraints while witnessing an integrality gap of at least 11/10 under the shortest-path graphic metric. The formalization depends only on the standard Lean 4 axioms (`propext`, `Quot.sound`, `Classical.choice`) and is archived in the Palomar Registry (`PALOMAR-2026-10-30-000013`).

1 Introduction

For a finite complete graph K_n with symmetric edge weights satisfying the triangle inequality, the Traveling Salesperson Problem (TSP) asks for a Hamiltonian cycle of minimum total weight. Although metric TSP remains NP-hard and cannot be approximated within 123/122 unless P = NP [1,5], two foundational results from the 1970s govern its approximation theory:

1. *Combinatorial upper bound:* The algorithm discovered independently by Christofides [2] and Serdyukov [7] combines a minimum spanning tree T with a minimum-weight perfect matching M on the odd-degree vertices of T, shortcuts an Eulerian circuit of the multigraph $T \uplus M$, and produces a tour of cost at most $\frac{3}{2}\,\text{OPT}$.

2. *Polyhedral lower bound:* The Held–Karp linear programming relaxation [3,4] enforces degree-2 conservation at every vertex and the Dantzig–Fulkerson–Johnson (DFJ) subtour elimination inequality $x(\delta(S)) \geq 2$ across every proper non-empty cut $\emptyset \subsetneq S \subsetneq V$. Even with all $2^n - 2$ subtour constraints present, the continuous polytope strictly contains the convex hull of integral tours for $n \geq 4$; on the 10-vertex Petersen graph P_{10}, the

fractional point $x_e = 2/3$ satisfies every DFJ constraint while admitting no Hamiltonian cycle.

Despite their compact textbook proofs, mechanizing these two arguments in an interactive theorem prover exposes several representation friction points. A single proof of the Christofides–Serdyukov bound must move between five different combinatorial objects: cyclic vertex permutations (for Hamiltonian tours and shortcutting), simple undirected edge sets (for spanning trees and matchings), edge multisets with parallel edges (when an edge of M duplicates an edge of T), closed multigraph walks (for Eulerian circuits), and subset-restricted subcycles (when projecting an optimal tour onto the odd-degree vertices of T). Similarly, proving the DFJ subtour inequality and separating the Held–Karp relaxation from the integral tour polytope requires bridging modular index wrap-around on lists with real-valued affine halfspaces and finite graph reflection.

This note reports on our Lean 4 formalization of both results, developed atop Mathlib [6].[1] Rather than formalizing imperative graph algorithms (such as Kruskal's algorithm or Edmonds' blossom shrinking), we focus on the combinatorial and polyhedral guarantees themselves, separating certificate properties from finite existence proofs. Specifically:

- In `Christofides.lean` (2,735 lines, 178 declarations), we formalize the complete 3/2-approximation proof for all $n \geq 3$, including the spanning-path lower bound $\text{cost}(T) \leq \text{OPT}$, the handshaking parity of odd-degree vertices, the alternating matching decomposition yielding $2 \cdot \text{cost}(M) \leq \text{OPT}$, multigraph degree parity on $T \uplus M$, constructive Hierholzer cycle splicing for Eulerian circuits, and sublist walk contraction under the triangle inequality.
- In `TSPComplexity.lean` (Sections 5–6), we give an algebraic $\mathbb{Z}_2$ telescoping proof of the DFJ cut condition, prove that every integral tour embeds into the Held–Karp polytope with fractional objective bounded by its tour cost, and verify by kernel reflection (`decide`) that the uniform $x_e = 2/3$ assignment on P_{10} has zero slack on singleton cuts, slack at least 2/3 on all non-trivial cuts ($2 \leq |S| \leq 8$), and induces an unconditional 11/10 integrality gap under the shortest-path graphic metric.

The remainder of this note is organized around the design choices that kept the proofs elementary. Section 2 discusses the representation of cyclic tours, metric walk contraction, and the spanning-tree lower bound. Section 3 covers Euler's handshaking parity and the alternating cycle decomposition used for the 1/2-OPT matching bound. Section 4 explains the multigraph union $T \uplus M$, Hierholzer trail extension, and the unconditional 3/2 bound. Section 5 presents the $\mathbb{Z}_2$ DFJ parity proof and the Petersen P_{10} separation theorems, and Section 6 summarizes the formalization statistics and design trade-offs.

2 Tours, Metric Shortcutting, and Spanning Trees

2.1 Representing Tours and Metric Costs

We work over the canonical finite vertex type `Fin n`. Edge weights are modeled as functions `CostFunction n := Fin n → Fin n → ℕ`, accompanied by the metric specification:

```
structure IsMetricCost {n : Nat} (c : CostFunction n) : Prop where
  symm : forall u v, c u v = c v u
  zero_diag : forall u, c u u = 0
  triangle : forall u v w, c u w <= c u v + c v w
```

[1] Source repository: https://github.com/IanMcNelly/lean4-tsp-complexity-christofides. The Christofides–Serdyukov formalization (`Christofides.lean`) is independently verified and archived in the Palomar Registry under entry PALOMAR-2026-10-30-000013.

Using $\mathbb{N}$-valued edge weights avoids real-number ring conversions throughout the combinatorial stages of the proof; the cast to $\mathbb{R}$ is deferred until the final 3/2 ratio statement and the continuous Held–Karp linear program.

A key early decision was how to represent Hamiltonian tours and intermediate walks. Mathlib's `SimpleGraph.Walk` is a dependent inductive family indexed by its endpoints, where every step carries a proof of adjacency. In metric TSP, however, the host graph for shortcutting is the complete graph K_n, and two central operations (restricting an optimal tour to the subset O of odd-degree vertices, and removing duplicate vertices from an Eulerian circuit) delete arbitrary interior vertices from a sequence. On dependent walks, deleting interior vertices requires re-stitching adjacency proofs at every splice.

Instead, we represent walks and tours simply as lists `p : List (Fin n)`. For an open walk, the traversal cost `walkCost c p` sums $c(u, v)$ over consecutive pairs, while for a closed cycle, `cycleEdges p` zips `p` with its one-step rotation:

```
def cycleEdges {n : Nat} (p : List (Fin n)) : List (Fin n * Fin n) :=
  match p with
  | [] => []
  | x :: xs => p.zip (xs ++ [x])

def tourCost {n : Nat} (c : CostFunction n) (p : List (Fin n)) : Nat :=
  ((cycleEdges p).map (fun e => c e.1 e.2)).sum

def IsTour {n : Nat} (G : Graph n) (p : List (Fin n)) : Prop :=
  p.length = n /\ p.Nodup /\ (forall e in cycleEdges p, G.adj e.1 e.2)
```

By separating the data (`List (Fin n)`) from the validity predicate (`IsTour`), any list-filtering operation immediately inherits Mathlib's `List.Sublist` API. In particular, metric walk contraction reduces to a single inductive lemma (`walkCost_sublist_le`): if `w1 <+ w2` is a sublist of an open walk, then under `IsMetricCost c`,

$$\text{walkCost}(c, w_1) \leq \text{walkCost}(c, w_2). \tag{1}$$

Both shortcutting steps in Christofides' algorithm (projecting the optimal tour onto O via `List.filter` and deduplicating the Eulerian circuit via `List.eraseDups`) are instances of `List.Sublist`, allowing us to reuse the same triangle-inequality induction in both places.

To avoid non-constructive Hilbert ε-selection when referring to the optimal tour value OPT, we define `allTours n := (List.finRange n).permutations` and compute `optTourCost c` directly by folding `min` over the mapped tour costs. For $n \geq 3$, the identity permutation `List.finRange n` witnesses that the tour set on `completeGraph n` is non-empty, giving an explicit optimal tour p^* achieving `tourCost c p* = optTourCost c` (`exists_optTour`).

2.2 Spanning Trees and the MST Lower Bound

While tours have a natural cyclic order, spanning trees and matchings are unordered edge sets. We represent undirected edges using the symmetric square `Sym2 (Fin n)`, written `s(u, v)`, and lift the cost function symmetrically via `undirectedEdgeCost c s(u, v) = min (c u v) (c v u)` (which equals `c u v` under `IsMetricCost c`). A spanning tree on `Fin n` is packaged as a finite edge set of size $n - 1$ that is loopless and connected under the reflexive-transitive closure `ConnectedInEdges`:

```
structure SpanningTree (n : Nat) where
  edges : Finset (Sym2 (Fin n))
  card_edges : edges.card = n - 1
  connected : forall u v : Fin n, ConnectedInEdges edges u v
  loopless : IsLooplessEdges edges
```

To prove the spanning-tree lower bound $\mathrm{cost}(T_{\mathrm{mst}}) \le \mathrm{OPT}$, textbook proofs delete a single edge from an optimal tour p^* to obtain a Hamiltonian path, which is a spanning tree of cost at most OPT. With our list representation, dropping the wrap-around edge of $p^* = x :: xs$ is immediate: `cycleEdges (x :: xs)` decomposes into the open path edges `directedPathEdges (x :: xs)` plus the single closing edge $((x :: xs).\mathrm{getLast}, x)$ (`cycleEdges_cons`). Mapping `Sym2.mk` over the open path produces an $(n-1)$-element duplicate-free edge finset `tourToSpanningPath` that connects the head x to every vertex in p^*, and hence connects all pairs in `Fin n` by transitivity. Because edge weights are non-negative, dropping the closing edge cannot increase total weight (`tourToSpanningPath_cost_le_tourCost`), yielding:

Theorem 2.1 (`MSTCost_le_OPT`)**.** *For $n \ge 3$, any cost function c, and any spanning tree T_{mst} satisfying `IsMST c` T_{mst},*

$$\mathrm{edgeSetCost}(c, T_{\mathrm{mst}}.\mathrm{edges}) \le \mathrm{optTourCost}(c).$$

3 Odd-Degree Parity and the 1/2-OPT Matching Bound

3.1 Incidence Double-Counting and Handshaking Parity

Let $O = \mathrm{oddDegreeVertices}(T.\mathrm{edges})$ be the subset of vertices having odd degree in the spanning tree T. To guarantee that O admits a perfect matching, one must first show that $|O|$ is even. Rather than deriving this from Mathlib's `SimpleGraph.sum_degrees_eq_twice_card_edges` (which would require translating back and forth between `Finset (Sym2 (Fin n))` and `SimpleGraph (Fin n)`), we prove the incidence double-counting identity directly on loopless `Sym2` finsets.

For each loopless edge $e = s(u, v)$ with $u \ne v$, filtering `Finset.univ` by membership $w \in e$ yields the two-element set $\{u, v\}$. Summing vertex degrees $\deg_E(v) = |\{e \in E \mid v \in e\}|$ over $v \in \mathrm{Fin}\ n$ and exchanging the order of summation gives $\sum_v \deg_E(v) = 2|E|$. Splitting the vertex sum across O and its complement O^c shows that $\sum_{v \in O} \deg_E(v)$ is even; since every term in the sum over O is congruent to 1 (mod 2), the cardinality $|O|$ must be even:

Theorem 3.1 (`even_card_oddDegreeVertices`)**.** *For any loopless undirected edge set E : $\mathrm{Finset}(\mathrm{Sym2}(\mathrm{Fin}\ n))$, the odd-degree vertex subset $\mathrm{oddDegreeVertices}(E)$ has even cardinality.*

3.2 Alternating Decomposition of the Shortcut Cycle

The most delicate part of the Christofides–Serdyukov proof is the bound $\mathrm{cost}(M) \le \frac{1}{2}\,\mathrm{OPT}$ for a minimum-weight perfect matching M on O. We define a perfect matching on an arbitrary vertex subset $S \subseteq \mathrm{Fin}\ n$ as a loopless edge finset whose endpoints lie in S and in which every vertex $v \in S$ is incident to exactly one edge:

```
structure PerfectMatching {n : Nat} (S : Finset (Fin n)) where
  edges : Finset (Sym2 (Fin n))
  endpoints_in : forall e in edges, forall v in e, v in S
  incident_once : forall v in S, (edges.filter (fun e => v in e)).card = 1
  loopless : IsLooplessEdges edges
```

Double-counting incidences across S gives $2|M.\mathrm{edges}| = |S|$ (`perfectMatching_card_two_mul`).

Given an optimal tour p^* on K_n, we project p^* onto O by filtering the list:

$$\mathrm{shortcutToSubset}(p^*, O) := p^*.\mathrm{filter}(\cdot \in O).$$

Because p^* is a permutation of Fin n, filtering by O preserves duplicate-freeness (`List.Nodup.filter`) and produces a cycle C_O of length $|O| = 2k$. By repeated application of the metric deletion step `tourCost_delete_head_step`, removing vertices outside O never increases cyclic traversal cost (`shortcutToSubset_cost_le_opt`):

$$\text{tourCost}(c, C_O) \leq \text{tourCost}(c, p^*) = \text{optTourCost}(c). \tag{2}$$

When O is non-empty ($|O| = 2k \geq 2$), the even-length cycle $C_O = [v_0, v_1, \ldots, v_{2k-1}]$ decomposes into two complementary perfect matchings on O. In Lean, instead of indexing into C_O with modular arithmetic ($2i \bmod 2k$), we define a recursive pairing function `cyclePairs` that consumes two vertices at a time from the head of a list, and obtain the second matching by applying the same function to the one-step cyclic rotation `cycleRotated`:

```
def cyclePairs {a : Type*} : List a -> List (Sym2 a)
  | [] => []
  | [_] => []
  | u :: v :: rest => s(u, v) :: cyclePairs rest

def cycleRotated {a : Type*} : List a -> List a
  | [] => []
  | u :: rest => rest ++ [u]
```

This formulation avoids modular index arithmetic entirely. When L is duplicate-free and has even length, structural induction consuming two elements at a time (`u :: v :: rest`) proves that `cyclePairs L` has no duplicate edges, covers every vertex in L with incidence multiplicity 1 (`cyclePairs_incident_once`), and therefore defines a valid `PerfectMatching` on L.toFinset (`evenCycleMatchingOne`). Since `cycleRotated L` preserves length, duplicate-freeness, and vertex support, applying `cyclePairs` to `cycleRotated L` immediately yields the second matching (`evenCycleMatchingTwo`) without duplicating the incidence proof.

Moreover, for any list $u :: v ::$ rest of even length, an inductive zip identity (`walk_alternating_pairs_cost`) establishes that the sum of the undirected pair weights of L and cycleRotated(L) equals the cyclic tour cost of L (`cycle_matchings_cost_sum`):

$$\begin{aligned} &\text{edgeSetCost}(c, M_1.\text{edges}) + \text{edgeSetCost}(c, M_2.\text{edges}) \\ &\quad = \text{tourCost}(c, C_O). \end{aligned} \tag{3}$$

Since M minimizes weight over all perfect matchings on O, we have $\text{cost}(M) \leq \text{cost}(M_1)$ and $\text{cost}(M) \leq \text{cost}(M_2)$. Adding the two inequalities and chaining with $\text{tourCost}(c, C_O) \leq \text{optTourCost}(c)$ discharges the matching bound in exact integer arithmetic:

Theorem 3.2 (`matchingCost_le_half_OPT`)**.** *For $n \geq 3$, any metric cost function c, any spanning tree T, and any minimum-weight perfect matching M on* oddDegreeVertices(T.edges)*,*

$$2 \cdot \text{edgeSetCost}(c, M.\text{edges}) \leq \text{optTourCost}(c).$$

4 Eulerian Multigraphs, Hierholzer Splicing, and the 3/2 Guarantee

4.1 Multigraph Union and Degree Parity

An edge of the matching M on O may already belong to the spanning tree T. If one took the set union T.edges $\cup$ M.edges as a `Finset`, any shared edge would collapse to multiplicity 1, destroying the even-degree invariant at its endpoints. We therefore define the multigraph union in `MultigraphEdges n := Multiset (Sym2 (Fin n))` using multiset addition:

$$\text{multigraphUnion}(E_1, E_2) := E_1.\text{val} + E_2.\text{val}. \tag{4}$$

Because multiset filtering is additive (`Multiset.filter_add`), the multigraph degree satisfies $\deg_{T \uplus M}(v) = \deg_T(v) + \deg_M(v)$ (`multigraphDegree_add`). If $v \in O$, then $\deg_T(v)$ is odd and $\deg_M(v) = 1$; if $v \notin O$, then $\deg_T(v)$ is even and $\deg_M(v) = 0$. In both cases, $\deg_{T \uplus M}(v)$ is even (`multigraph_degree_even`).

An Eulerian circuit of a multigraph E is specified as a closed vertex walk whose induced undirected edge multiset equals E and which visits every vertex:

```
structure EulerianCircuit {n : Nat} (E : MultigraphEdges n) where
  walk : List (Fin n)
  is_closed : walk.head? = walk.getLast?
  edge_multiset_eq : (pathEdges walk : Multiset (Sym2 (Fin n))) = E
  visits_all : forall v : Fin n, v in walk
```

Because `edge_multiset_eq` equates the multiset of walk edges with $T.\mathrm{val} + M.\mathrm{val}$, multiset map summation shows that the traversal cost of any Eulerian circuit W of $T \uplus M$ equals the sum of the tree and matching weights (`circuitCost_eq_treeCost_add_matchingCost`):

$$\begin{aligned} \mathrm{walkCost}(c, W.\mathrm{walk}) = {} & \mathrm{edgeSetCost}(c, T.\mathrm{edges}) \\ & + \mathrm{edgeSetCost}(c, M.\mathrm{edges}). \end{aligned} \tag{5}$$

4.2 Constructive Existence via Hierholzer Cycle Splicing

To obtain an unconditional approximation theorem (rather than assuming an `EulerianCircuit` as an external hypothesis), Section 20 of `Christofides.lean` proves that every connected, loopless, even-degree multigraph of the form $E_1 \uplus E_2$ admits an Eulerian circuit (`exists_eulerianCircuit`). The proof formalizes Hierholzer's cycle-splicing argument in three stages:

1. *Walk endpoint parity (`walk_degree_mod_two`):* For any loopless open walk $u :: ws$ with terminal vertex b, the multigraph degree of any vertex v in $\mathrm{pathEdges}(u :: ws)$ modulo 2 equals $(\mathbf{1}_{v=u} + \mathbf{1}_{v=b}) \bmod 2$.

2. *Fuel-bounded trail extension (`extend_trail_to_cycle`):* Given a residual even-degree loopless multigraph R and a non-empty trail $u :: ws$ using a submultiset of R, if the current endpoint b is distinct from u, then $\mathrm{pathEdges}(u :: ws)$ uses an odd number of edges incident to b. Since b has even degree in R, the difference multigraph $R - \mathrm{pathEdges}(u :: ws)$ has positive degree at b, providing an unused incident edge $s(b, z)$ (`exists_incident_edge_of_pos_degree`). Induction on the remaining edge budget $|R| - |\mathrm{pathEdges}(u :: ws)|$ extends the trail until it closes at u.

3. *Cycle splicing (`eulerianWalkInduction_holds`):* By strong induction on the number of unused edges in $H = T \uplus M$, if a closed sub-walk W_0 does not yet exhaust H, connectivity of T locates a vertex $v \in W_0$ incident to an unused edge in $H - \mathrm{pathEdges}(W_0)$. Extending a closed cycle from v inside the even-degree residual multigraph $H - \mathrm{pathEdges}(W_0)$ and splicing it into W_0 via `pathEdges_append_cons` strictly increases the number of covered edges while preserving closure.

Similarly, the existence of a minimum spanning tree (`exists_mst`) and a minimum-weight perfect matching on O (`exists_minWeightMatching`) follow from `Nat.find` minimization (`exists_min_nat`) once each type is shown to be inhabited (by `tourToSpanningPath` and `evenCycleMatchingOne`, respectively). Combining these three existence theorems inhabits `ChristofidesCertificate n c` for every $n \geq 3$ (`exists_christofidesCertificate`).

4.3 Shortcutting the Circuit and Assembling the Ratio

Given an Eulerian circuit W, we define `shortcutCircuit w := (w.dropLast).eraseDups`. Dropping the duplicated closing vertex before calling `List.eraseDups` ensures that the resulting list is duplicate-free (`List.nodup_eraseDups`), contains every vertex of Fin n (since W visits all vertices), and therefore forms a valid Hamiltonian tour of `completeGraph n` (`shortcutCircuit_isTour`). Because `w.eraseDups` is a sublist of w beginning at the same head vertex, the sublist contraction lemma implies

$$\text{tourCost}(c, \text{shortcutCircuit}(W.\text{walk})) \leq \text{walkCost}(c, W.\text{walk})$$

(`shortcut_tourCost_le_circuitCost`). Combining this with the tree and matching bounds yields both the integer-scaled and real-valued unconditional approximation guarantees:

Theorem 4.1 (`christofides_certificate_approx_ratio_unconditional`)**.** *For every $n \geq 3$ and every metric cost function c : CostFunction n, there exists a certificate* cert : ChristofidesCertificate n c *whose shortcut tour π =* christofidesTour(cert) *is a valid Hamiltonian tour of K_n satisfying*

$$2 \cdot \text{tourCost}(c, \pi) \leq 3 \cdot \text{optTourCost}(c),$$
$$(\text{tourCost}(c, \pi) : \mathbb{R}) \leq \frac{3}{2} (\text{optTourCost}(c) : \mathbb{R}).$$

5 The Held–Karp Polytope and the Petersen P_{10} Integrality Gap

A natural companion question to the 3/2 upper bound is why standard linear programming relaxations do not directly yield integral tours. In Sections 5 and 6 of `TSPComplexity.lean`, we formalize the Dantzig–Fulkerson–Johnson (DFJ) subtour elimination constraints, the continuous Held–Karp relaxation, and the 10-vertex Petersen graph counterexample P_{10}.

5.1 A $\mathbb{Z}_2$ Telescoping Proof of the DFJ Cut Bound

For a proper non-empty vertex subset $\emptyset \subsetneq S \subsetneq$ Fin n, the DFJ subtour inequality asserts that every Hamiltonian tour p crosses the cut boundary $\delta(S)$ at least twice. Formalizing "a closed loop that visits both S and S^c must cross $\delta(S)$ an even number of times" by inducting on list segments is awkward because of the wrap-around step from index $n-1$ to 0.

We avoid case-splitting on wrap-around indices by working algebraically over Fin n. First, we equip Fin n ($n \geq 1$) with the cyclic shift bijection `cycleNextEquiv : Fin n ≃ Fin n` mapping $i \mapsto (i+1) \bmod n$. Given the Boolean cut-membership sequence $b(i) = \text{decide}(p[i] \in S)$ and its $\{0,1\} \subset \mathbb{Z}$ encoding $\chi(i) = \text{boolToInt}(b(i))$, we define the signed cyclic difference $\Delta_i = \chi(\text{cycleNext } i) - \chi(i) \in \{-1, 0, 1\}$. Re-indexing the first sum along `cycleNextEquiv` (`Equiv.sum_comp`) immediately proves that the signed differences telescope to zero around the cycle (`cycle_delta_sum_zero`):

$$\sum_{i \in \text{Fin } n} \Delta_i = \sum_{i \in \text{Fin } n} \chi(\text{cycleNext } i) - \sum_{i \in \text{Fin } n} \chi(i) = 0 \in \mathbb{Z}. \tag{6}$$

On the other hand, $|\Delta_i| = 1$ if and only if step i crosses $\delta(S)$ ($b(i) \neq b(\text{cycleNext } i)$), and $|\Delta_i| = 0$ otherwise (`abs_cycleDelta_eq_ite`). Because $|z| \equiv z \pmod 2$ for every integer $z \in \{-1, 0, 1\}$ (discharged by `decide` on the four Boolean pairs in `cycleDelta_zmod_two`),

projecting the sum to `ZMod 2` shows that the number of boundary crossings is always even (`cyclic_cut_crossings_even`):

$$\begin{aligned}&(|\{i \in \mathrm{Fin}\ n \mid b(i) \neq b(\mathrm{cycleNext}\ i)\}| : \mathrm{ZMod}\ 2)\\ &\quad = \sum_{i \in \mathrm{Fin}\ n} |\Delta_i| = \sum_{i \in \mathrm{Fin}\ n} \Delta_i = 0.\end{aligned} \tag{7}$$

Since S is both non-empty and proper, and p visits every vertex of Fin n, the sequence b attains both `true` and `false`. A circular Boolean sequence with zero boundary transitions is constant by simple induction (`bool_eq_zero_of_no_boundary`), so b has at least one transition; by evenness, it has at least two (`cyclic_boolean_transitions_ge_two`). Mapping the two distinct transition indices into `cycleEdges p` proves the DFJ cut theorem:

Theorem 5.1 (`dfj_subtour_cut_bound`)**.** *For $n \geq 3$, any valid Hamiltonian tour p on G, and any subset $S \subseteq$ Fin n with S.*Nonempty *and $S \neq$* Finset.univ*, at least two edges of p cross S:*

$$2 \leq (\mathrm{tourCutEdges}\ p\ S).\mathrm{length}.$$

5.2 Embedding Tours into the Held–Karp Polytope

A point in the Held–Karp subtour relaxation on n vertices is a symmetric matrix $x : \mathrm{Fin}\ n \to \mathrm{Fin}\ n \to \mathbb{R}$ with zero diagonal, entries in $[0, 1]$, degree-2 conservation $\sum_{u \neq v} x_{vu} = 2$ at every vertex v, and cut capacity $\mathrm{cutCapacity}(x, S) := \sum_{u \in S} \sum_{v \notin S} x_{uv} \geq 2$ across every proper non-empty cut S (`HeldKarpPoint n`). The fractional objective is $\mathrm{fractionalCost}(c, x) = \frac{1}{2} \sum_{u,v} c(u, v) x_{uv}$, where the factor of $1/2$ accounts for each undirected edge appearing twice in the ordered double sum.

For any valid tour p on $n \geq 3$ vertices, its symmetric indicator matrix $\mathrm{tourPoint}(p)(u, v)$ assigns 1 if (u, v) or (v, u) belongs to $\mathrm{cycleEdges}(p)$ and 0 otherwise. Using the inverse shift `cyclePrev` and the inequality $\mathrm{cycleNext}\ k \neq \mathrm{cyclePrev}\ k$ for $n \geq 3$ (`cycleNext_ne_cyclePrev`), each vertex $v = p[k]$ has exactly two distinct incident neighbours $p[\mathrm{cycleNext}\ k]$ and $p[\mathrm{cyclePrev}\ k]$ along the tour, establishing degree-2 conservation (`tourPoint_degree_two`). Combining this with the two distinct boundary crossings from Section 5.1 proves that $\mathrm{tourPoint}(p)$ is a feasible Held–Karp point (`tour_is_held_karp_point`), and that for any symmetric cost function c,

$$\mathrm{fractionalCost}(c, \mathrm{tourPoint}(p)) \leq (\mathrm{tourCost}(c, p) : \mathbb{R})$$

(`fractionalCost_tourPoint_le_tourCost`).

5.3 Separating the Held–Karp Relaxation on P_{10}

To show that the Held–Karp relaxation strictly contains the integral tour polytope

$$\begin{aligned}\mathrm{TSPPolytope}(10) := \mathrm{convexHull}\ \mathbb{R}\ \big\{&\mathrm{tourPoint}(p) \mid\\ &p.\mathrm{length} = 10 \wedge p.\mathrm{Nodup}\big\},\end{aligned}$$

we formalize the 10-vertex Petersen graph P_{10} (`petersenAdj` on `Fin 10`, decomposing vertices into an outer 5-cycle $\{0, \ldots, 4\}$, radial spokes, and an inner pentagram $\{5, \ldots, 9\}$) and define the uniform fractional point:

$$\mathrm{petersenFractional}(u, v) := \begin{cases} 2/3 & \text{if } \mathrm{petersenAdj}(u, v),\\ 0 & \text{otherwise.}\end{cases} \tag{8}$$

Because Fin 10 is finite, purely combinatorial facts about P_{10} can be discharged inside the Lean 4 kernel by decidable reflection (`decide`), without invoking the compiler-trust axiom `native_decide`:

- *Feasibility and DFJ slack profile:* Kernel evaluation confirms that every vertex of P_{10} has degree 3, every singleton cut $\{v\}$ crosses exactly 3 edges (`petersen_singleton_cut_card_three`), and every non-trivial cut S with $2 \le |S| \le 8$ crosses at least 4 edges because P_{10} has girth 5 (`petersen_nontrivial_cut_card_ge_four`). Scaling by 2/3 via `linarith` proves that `petersenFractional` is a valid `HeldKarpPoint 10` (`petersenHK`), has zero DFJ slack on all 10 singleton cuts (`petersen_dfjSlack_singleton_zero`), and has strictly positive interior slack
$$\mathrm{dfjSlack}(\mathrm{petersenFractional}, S) \ge 4 \cdot \frac{2}{3} - 2 = \frac{2}{3}$$
across all non-trivial cuts (`petersen_dfjSlack_nontrivial_ge_two_thirds`).

- *Non-Hamiltonicity and hyperplane separation:* Rather than testing all $10! = 3{,}628{,}800$ permutations of Fin 10, we define `petersenWalks k`, which inductively extends duplicate-free P_{10}-walks of length $k+1$. Because P_{10} is 3-regular and duplicate-free walks cannot revisit vertices, `petersenWalks 9` contains only 240 simple paths of length 10, and `decide` verifies in milliseconds that none of them closes into a 10-cycle (`petersen_no_closed_walk_10`, `petersen_no_hamiltonian_tour`). Consequently, any 10-vertex tour p must use at least one non-edge of P_{10}. Under the separating linear functional
$$\mathrm{petersenSepCost}(x) := \sum_{u,v \in \mathrm{Fin}\ 10} \mathbf{1}_{\neg \mathrm{petersenAdj}(u,v)}\, x_{uv},$$
every integral tour indicator satisfies $\mathrm{petersenSepCost}(\mathrm{tourPoint}(p)) \ge 1$ (`petersen_tour_cost_ge_one`), so the entire convex hull TSPPolytope(10) lies in the closed halfspace $\{x \mid 1 \le \mathrm{petersenSepCost}(x)\}$ by Mathlib's `convexHull_min`. Since $\mathrm{petersenSepCost}(\mathrm{petersenFractional}) = 0$, this strictly separates the Held–Karp relaxation from the TSP polytope (`petersen_strictly_separates_held_karp`).

- *The* 11/10 *graphic metric integrality gap:* Finally, we define the shortest-path graphic metric $c_{P_{10}} := \mathrm{petersenMetricCost}$ on P_{10} (which has diameter 2):
$$c_{P_{10}}(u,v) := \begin{cases} 0 & \text{if } u = v, \\ 1 & \text{if } \mathrm{petersenAdj}(u,v), \\ 2 & \text{otherwise.} \end{cases} \tag{9}$$
Kernel verification over $(\mathrm{Fin}\ 10)^3$ confirms the triangle inequality (`petersenMetricCost_isMetric`). On the 15 edges of P_{10} (each of weight 1), `petersenFractional` achieves Held–Karp objective $15 \times \frac{2}{3} = 10$. Conversely, because every 10-tour p traverses 10 edges of cost at least 1 and at least one non-edge e^* of cost 2 (isolated via `List.append_of_mem`), every integral tour has cost at least $9 \times 1 + 2 = 11$ (`petersen_metric_tourCost_ge_eleven`):

Theorem 5.2 (`petersen_heldKarp_metric_integrality_gap`)**.** *Under the metric cost function* $c_{P_{10}} := \mathrm{petersenMetricCost}$*, the Held–Karp relaxation admits a feasible point of fractional cost at most* 10*, whereas every Hamiltonian tour on* Fin 10 *has cost at least* 11*:*
$$\begin{aligned} &(\exists\, \mathrm{hk} : \mathrm{HeldKarpPoint}\ 10,\ \mathrm{fractionalCost}(c_{P_{10}}, \mathrm{hk}.x) \le 10) \\ &\quad \wedge \Big(\forall p : \mathrm{List}(\mathrm{Fin}\ 10),\ p.\mathrm{length} = 10 \to p.\mathrm{Nodup} \\ &\qquad \to 11 \le \mathrm{tourCost}(c_{P_{10}}, p)\Big). \end{aligned}$$

Table 1: Summary of principal mechanized theorems across `Christofides.lean` and `TSPComplexity.lean` (Sections 5–6), verified under Lean 4 (`v4.35.0-rc2`) with zero `sorry`s.

Declaration Name	Module	Mathematical Statement	Key Proof Technique
`MSTCost_le_OPT`	`Christofides`	$\mathrm{cost}(T_{\mathrm{mst}}) \le \mathrm{OPT}$ $(n \ge 3)$	Delete wrap-around tour edge (`tourToSpanningPath`)
`even_card_oddDegreeVertices`	`Christofides`	$2 \mid \lvert\mathrm{oddDegreeVertices}(E)\rvert$	Fiberwise `Sym2` incidence double-counting
`perfectMatching_card_two_mul`	`Christofides`	$2 \mid M.\mathrm{edges}\rvert = \lvert S\rvert$	Incidence partition over S
`cycle_matchings_cost_sum`	`Christofides`	$\mathrm{cost}(M_1) + \mathrm{cost}(M_2) = \mathrm{tourCost}(c, C_O)$	Alternating `cyclePairs` + `cycleRotated` induction
`matchingCost_le_half_OPT`	`Christofides`	$2 \cdot \mathrm{cost}(M) \le \mathrm{OPT}$	Filter p^* to O (`shortcutToSubset`) + parity split
`multigraph_degree_even`	`Christofides`	$\forall v,\ 2 \mid \deg_{T \uplus M}(v)$	Additive multiset degree on $T.\mathrm{val} + M.\mathrm{val}$
`exists_christofidesCertificate`	`Christofides`	$\exists\, \mathrm{cert} : \mathrm{ChristofidesCertificate}\ n\ c,\ \top$	`Nat.find` extrema + Hierholzer trail splicing
`shortcut_tourCost_le_circuitCost`	`Christofides`	$\mathrm{tourCost}(c, \mathrm{shortcutCircuit}\ W) \le \mathrm{walkCost}(c, W)$	Sublist contraction (`walkCost_sublist_le`)
`christofides_certificate_approx_ratio_unconditional`	`Christofides`	$\exists \mathrm{cert},\ \mathrm{IsTour}\ \pi \wedge \mathrm{cost}(\pi) \le \frac{3}{2}\mathrm{OPT}$	Synthesis of tree, matching, and shortcut bounds
`cyclic_cut_crossings_even`	`TSPComplexity`	$2 \mid \lvert\{i \mid b(i) \neq b(\mathrm{cycleNext}\ i)\}\rvert$	Telescoping sum along `cycleNextEquiv` in `ZMod 2`
`dfj_subtour_cut_bound`	`TSPComplexity`	$2 \le \lvert\delta(S) \cap E(p)\rvert$ $(\emptyset \subsetneq S \subsetneq V)$	Surjectivity of p + even boundary transitions
`tour_is_held_karp_point`	`TSPComplexity`	$\exists \mathrm{hk} : \mathrm{HeldKarpPoint}\ n,\ \mathrm{hk}.x = \mathrm{tourPoint}\ p$	Predecessor/successor distinctness + DFJ cut bound
`petersen_dfjSlack_nontrivial_ge_two_thirds`	`TSPComplexity`	$2 \le \lvert S\rvert \le 8 \implies \mathrm{dfjSlack}(x_{P_{10}}, S) \ge 2/3$	Kernel girth-5 cut reflection (≥ 4 edges) + `linarith`
`petersen_strictly_separates_held_karp`	`TSPComplexity`	$x_{P_{10}} \in P_{\mathrm{HK}}(10) \setminus \mathrm{TSPPolytope}(10)$	Depth-9 walk tree (`decide`) + `convexHull_min`
`petersen_heldKarp_metric_integrality_gap`	`TSPComplexity`	$\mathrm{LP}_{\mathrm{HK}}(P_{10}) \le 10 < 11 \le \mathrm{OPT}(P_{10})$	Diameter-2 graphic metric + non-edge split

6 Formalization Summary and Concluding Remarks

Table 1 lists the principal declarations from both modules. All theorems compile under Lean 4 (`v4.35.0-rc2`) and depend solely on the three standard axioms `propext`, `Quot.sound`, and `Classical.choice`.[2] Even the finite checks on P_{10} (evaluating all 1,022 proper cuts and the 240 simple walks of length 10) run inside the standard Lean kernel via `decide`, without relying on `Lean.ofReduceBool` or `native_decide`.

Two practical lessons stood out during the formalization:

1. *Choose data structures for the hardest transformation, not just the static definition.* Representing tours as `List (Fin n)` with an external `IsTour` predicate rather than dependent walks made both vertex-subset projection (`List.filter`) and Eulerian circuit shortcutting (`List.eraseDups`) reducible to a single 25-line inductive lemma on `List.Sublist`. Likewise, defining the two alternating matchings of an even cycle via a two-element list consumer (`cyclePairs`) and a head-to-tail rotation (`cycleRotated`) eliminated modular index arithmetic from the 1/2-OPT matching proof.

2. *Prune finite search trees before invoking kernel reflection.* A naive attempt to prove the non-Hamiltonicity of P_{10} by evaluating all $10! = 3{,}628{,}800$ vertex permutations inside the kernel times out. Generating only simple P_{10}-walks of length 10 (`petersenWalks 9`) shrinks the search space to 240 candidates, allowing `decide` to finish in milliseconds while remaining purely kernel-checked.

References

[1] D. L. Applegate, R. E. Bixby, V. Chvátal, and W. J. Cook. *The Traveling Salesman Problem: A Computational Study.* Princeton University Press, 2006.

[2] N. Christofides. Worst-case analysis of a new heuristic for the travelling salesman problem. Technical Report 388, Graduate School of Industrial Administration, Carnegie-Mellon University, 1976.

[3] G. Dantzig, R. Fulkerson, and S. Johnson. Solution of a large-scale traveling-salesman problem. *Journal of the Operations Research Society of America*, 2(4):393–410, 1954.

[2] In addition to the polyhedral proofs in Sections 5–6, `TSPComplexity.lean` contains auxiliary specification experiments (such as a fuel-bounded permutation frontier scanner, a conditional Yannakakis slack-factorization scaling wrapper, and a Hamiltonian Cycle to Decision TSP cost reduction) together with Python reference scripts in `benchmarks/`. We omit those auxiliary specifications here to focus on the unconditional approximation and polyhedral results.

[4] M. Held and R. M. Karp. The traveling-salesman problem and minimum spanning trees. *Operations Research*, 18(6):1138–1162, 1970.

[5] R. M. Karp. Reducibility among combinatorial problems. In *Complexity of Computer Computations*, pages 85–103. Plenum Press, 1972.

[6] The mathlib Community. The Lean mathematical library. In *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP)*, pages 367–381, 2020.

[7] A. I. Serdyukov. On some extremal walks in graphs. *Upravlyaemye Sistemy*, 17:76–79, 1978.